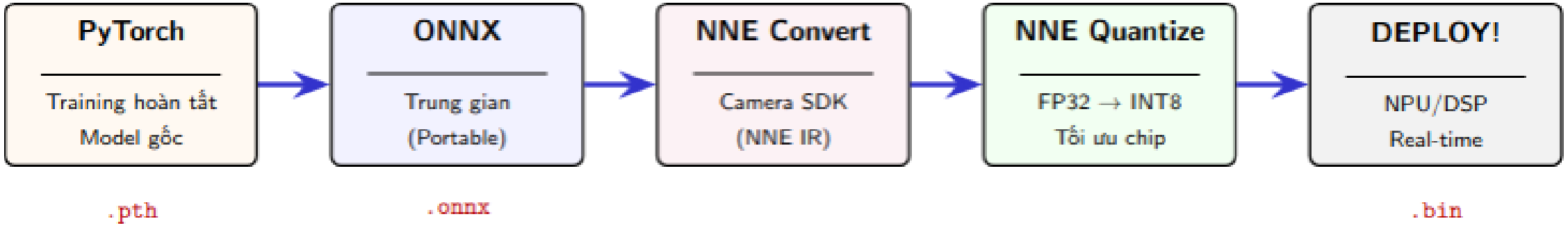
HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.

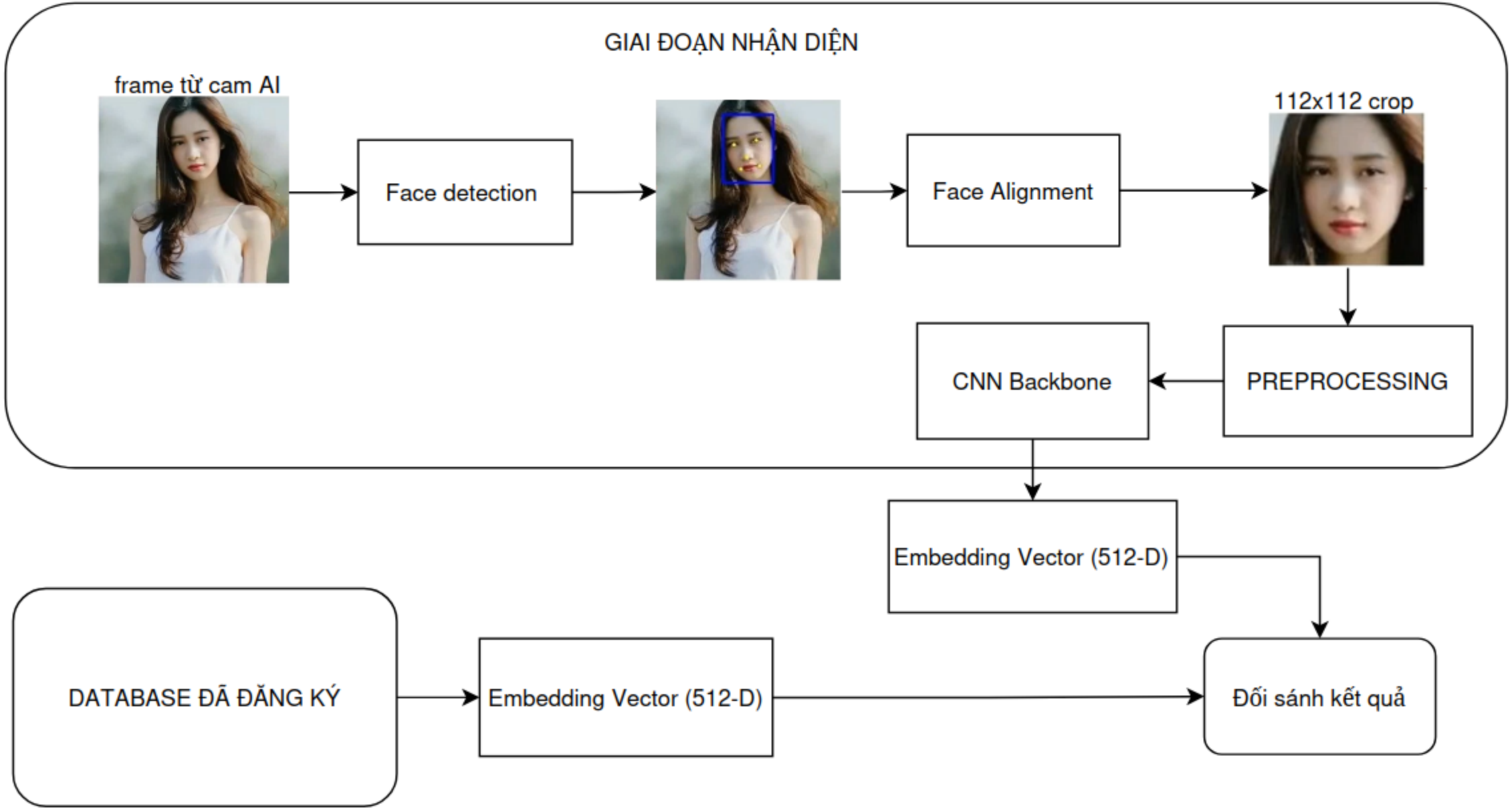
Nghiên cứu tính năng Face Regconition cho Indoor Advance camera



Đặt vấn đề- Bài toán Edge AI

- Camera: Thiết bị nhúng (SoC)
- RAM <256MB
- Flash Storage cực ít (<16MB), yêu cầu chạy offline & Real-time (<50ms)
- Cần một mô hình nhẹ (1-5MB) nhưng accuracy đủ tốt cho camera AI
- Hiện nay các bài toán xử lý ảnh thường dựa trên các mạng CNN là chủ yếu
- Cần tìm các model hiện có và tinh chỉnh trace off hợp lí để convert lên camera

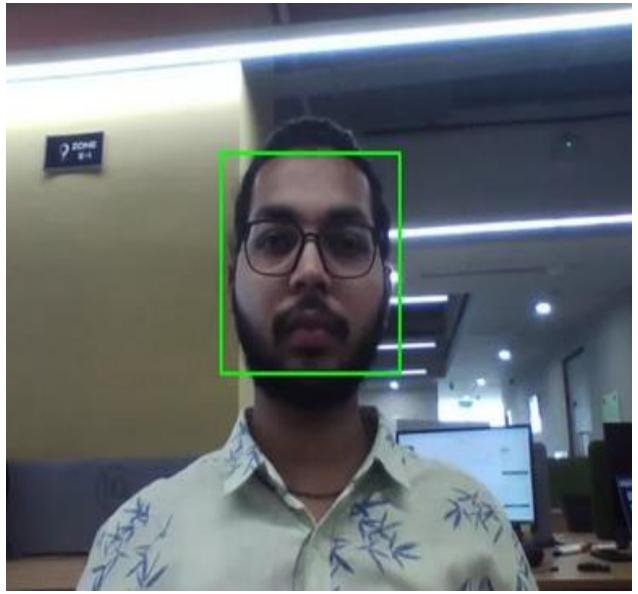
GENERAL PIPELINE INFERENCE



Face Detection

Input: frame ảnh người

Output: (x1,y1,x2,y2) hoặc (x,y,w,h) để hợp thành boundingbox, các bounding box này chứa mặt người bên trong từ đó giúp thực hiện crop face, từ đó trích xuất đặc trưng của ảnh



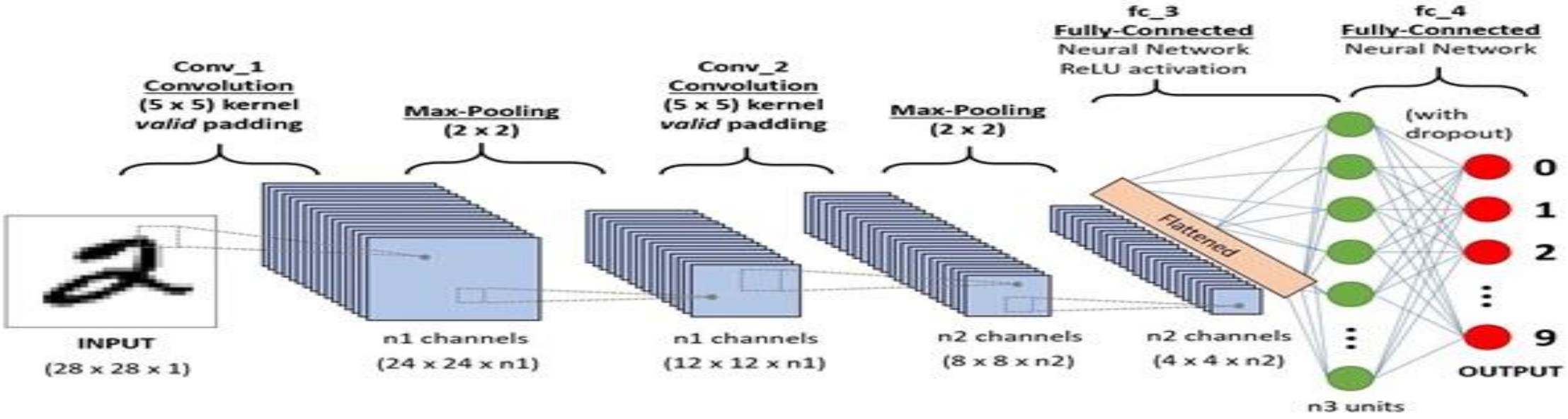
Face Alignment

Sau bước Face Detection -> các face chưa được alignment(căn chỉnh)

Áp dụng Face Alignment là cần thiết (eg: Buffalo_S ~10MB)

Overview về CNN

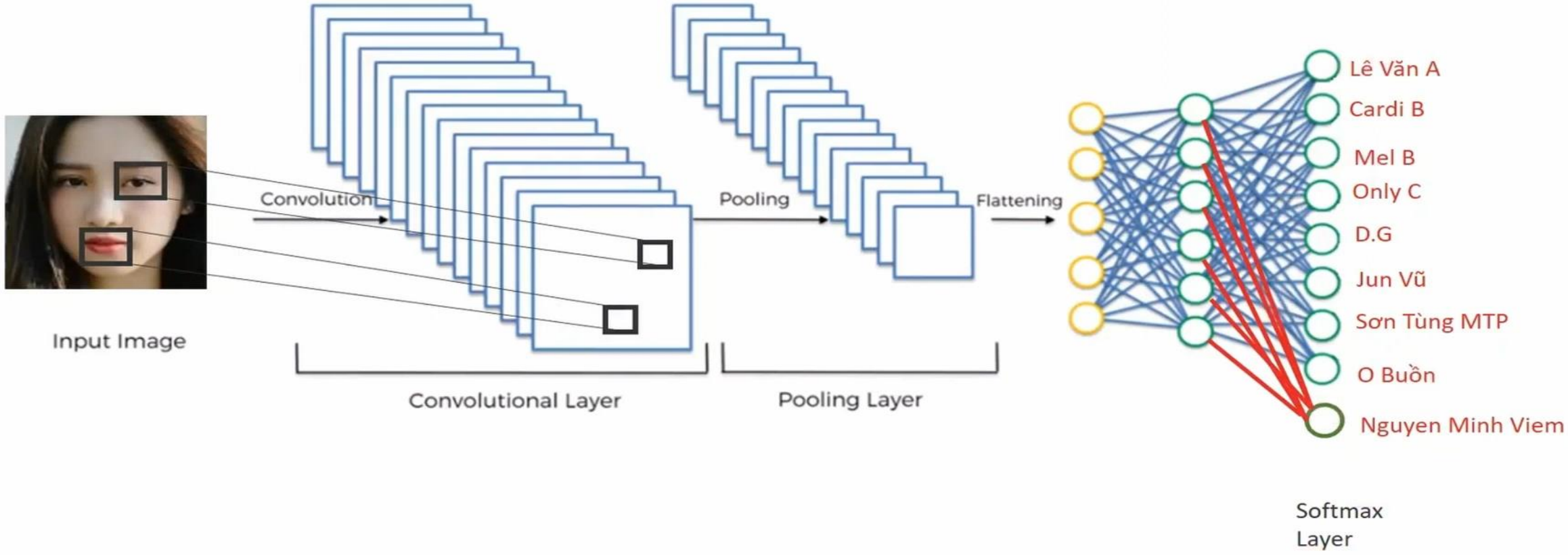
- Đầu vào của lớp tích chập là hình ảnh
- Chọn đối số, áp dụng các bộ lọc với các bước nhảy, padding nếu cần. Thực hiện tích chập cho hình ảnh và áp dụng hàm kích hoạt ReLU cho ma trận hình ảnh.
- Thực hiện Pooling để giảm kích thước cho hình ảnh.
- Thêm nhiều lớp tích chập sao cho phù hợp
- Xây dựng đầu ra và dữ liệu đầu vào thành 1 lớp được kết nối đầy đủ (Full Connected)
- Sử dụng hàm kích hoạt để tìm đối số phù hợp và phân loại hình ảnh.



Recognition Algorithms

Hướng 1: Đưa dataset vào một mạng CNN training vs labels có sẵn để cho ra một models có khả năng predict

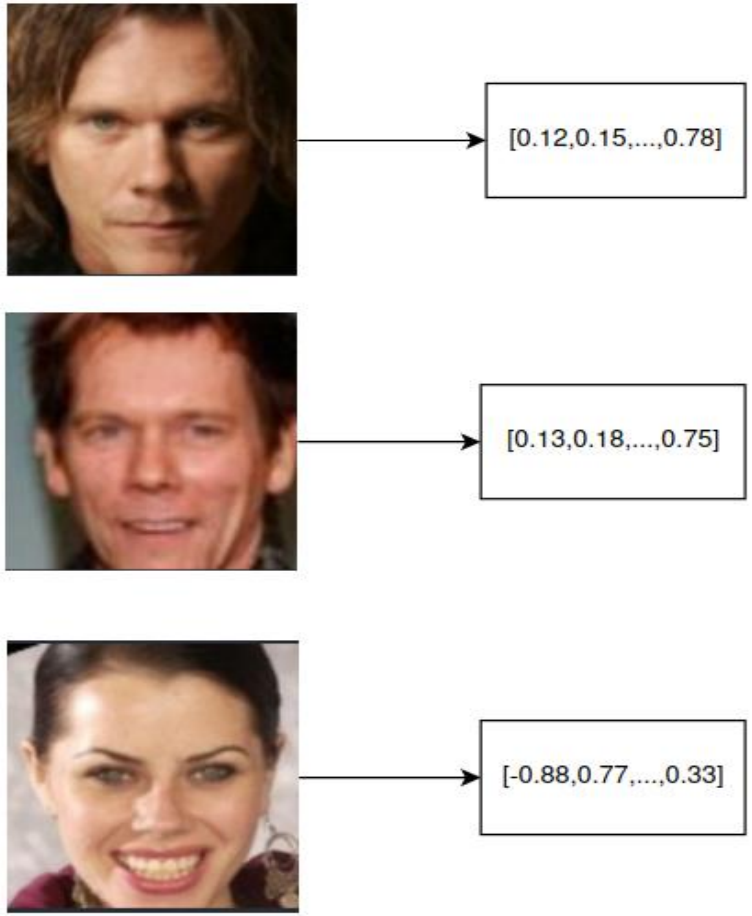
Hạn chế: khi cần thêm persons cần training lại toàn bộ mô hình, ta sẽ không đi theo hướng này (không giải quyết được bài toán one shot learning)



Learning Similarity

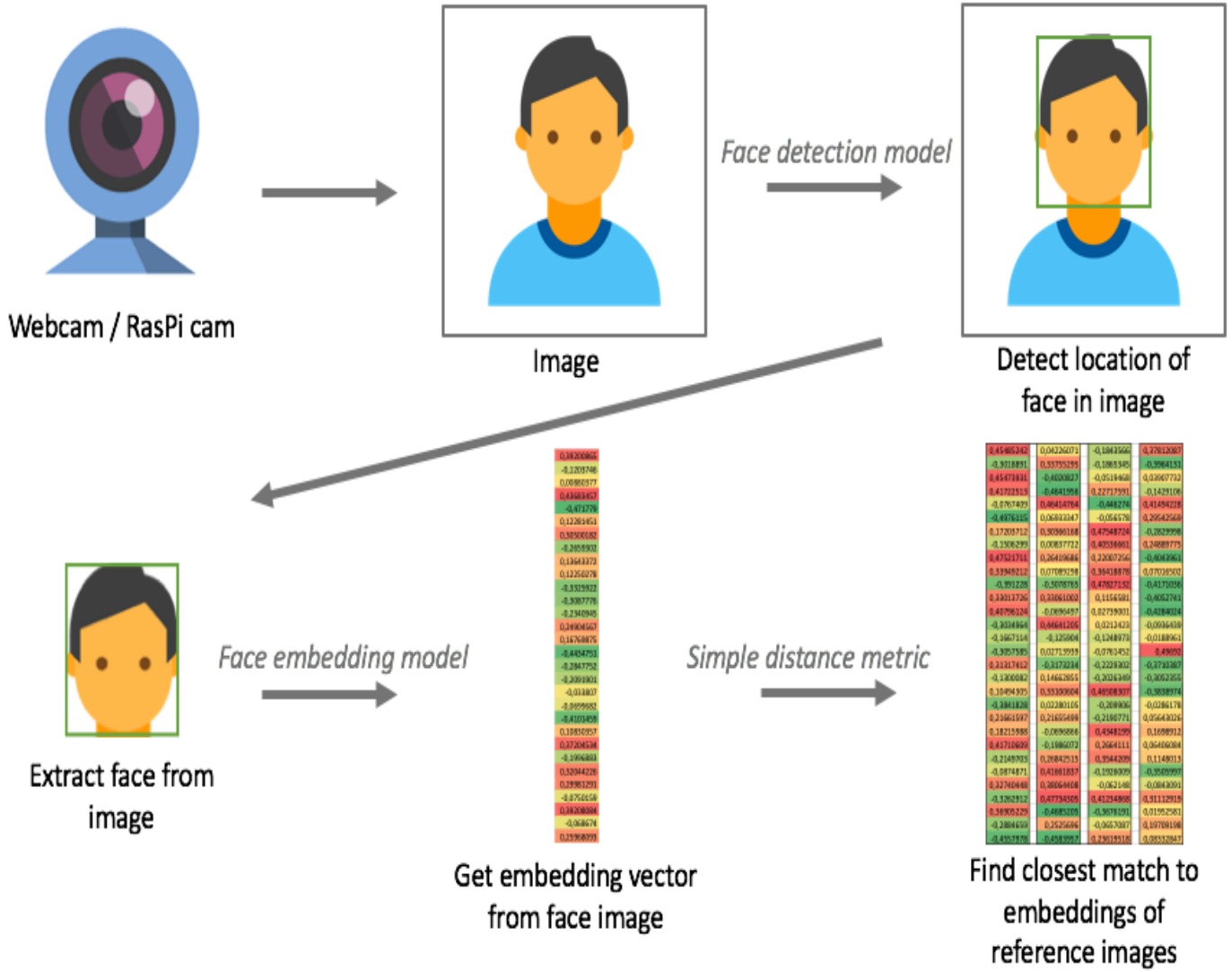
- Mô hình học cách đo độ giống nhau giữa từng người
- Output thường là:
  - Distance
  - Vector embedding
  - Cosine similarity

Trong Face Recognition ta nên dùng cosine



Face Representation

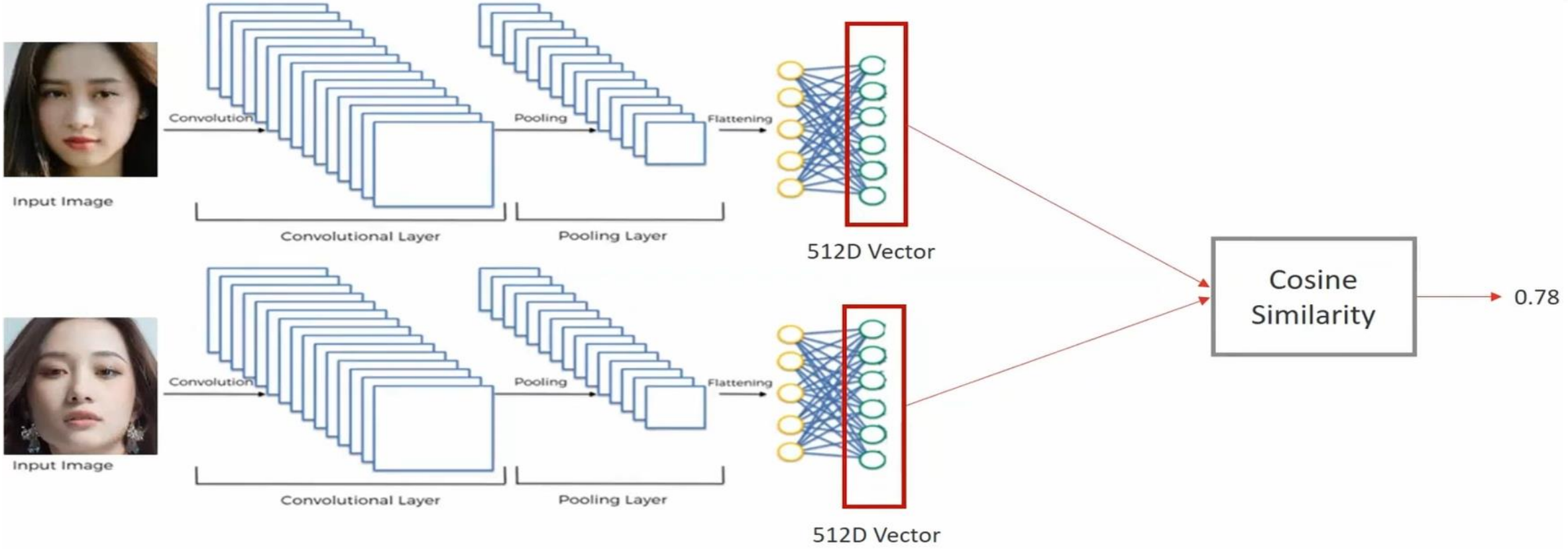
- Xây dựng một mô hình có khả năng trích xuất vecto đặc trưng của ảnh (có khả năng biểu diễn embedding space)
- Input: tensor (1,3,112,112)
- Output quy định là 1 vecto embedding thường là 512 chiều, số chiều tăng ảnh hưởng đến response khi inference



Recognition Algorithms

Hướng 2: : Learning similarity để giải quyết bài toán One-shot learning

Bài toán 1-N:Dùng một thuật toán search dựa trên độ similarity giữa các vector đặc trưng kết với công thức tính khoảng cách để cho được kết quả



Lựa chọn mô hình

- Qua quá trình survey -> lựa chọn mô hình dựa trên cấu trúc mạng CNN chuyên cho nhận diện khuôn mặt trên edge device MobileFaceNet với bacbone là cac mô hình MobileNet (hơn nữa MobileNet còn xuất hiện trong demos của toolchain)

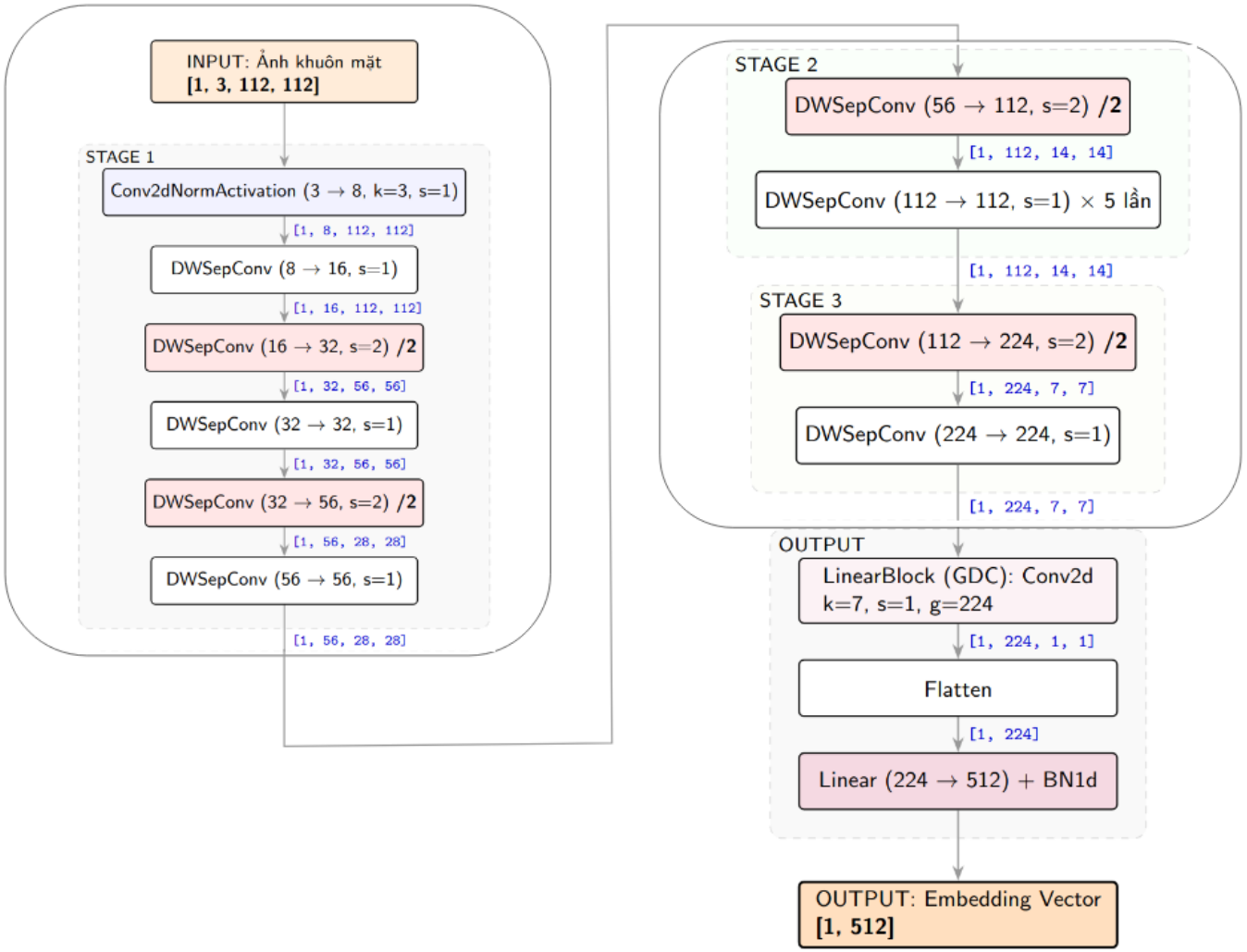
Model	MPARAMS	MFLOPs	LFW (%)	CA-LFW (%)	CP-LFW (%)	CFP-FP (%)	AgeDB-30 (%)	IJB-B (%)	IJB-C (%)
VarGFaceNet [12], [23]	5.0	1022	**99.85**	95.15	88.55	**98.50**	**98.15**	92.9	94.7
ShuffleFaceNet 2x [22]	4.5	1050	99.62	-	-	97.56	97.28	-	-
MixFaceNet-M [13]	3.95	626.1	99.68	-	-	-	97.05	91.55	93.42
ShuffleMixFaceNet-M [13]	3.95	626.1	99.60	-	-	-	96.98	91.47	91.47
MobileFaceNetV1 [12]	3.4	1100	99.4	94.47	87.17	95.8	96.4	92.0	93.9
ProxylessFaceNAS [12]	3.2	900	99.2	92.55	84.17	94.7	94.4	87.1	89.7
MixFaceNet-S [13]	3.07	451.7	99.6	-	-	-	96.63	90.17	92.30
ShuffleMixFaceNet-S [13]	3.07	451.7	99.58	-	-	-	97.05	90.94	93.08
ShuffleFaceNet 1.5x [12], [22]	2.6	577.5	99.7	95.05	88.50	96.9	97.3	92.3	94.3
MobileFaceNet [12]	2.0	933	99.7	95.2	89.22	96.9	97.6	92.8	94.7
PocketNetM-256 [14]	1.75	1099.15	99.58	95.63	90.03	**95.66**	**97.17**	90.74	92.70
PocketNetM-128 [14]	1.68	1099.02	99.65	95.67	90.00	95.07	96.78	90.63	92.63
MixFaceNet-XS [13]	1.04	161.9	99.60	-	-	-	95.85	88.48	90.73
ShuffleMixFaceNet-XS [13]	1.04	161.9	99.53	-	-	-	95.62	87.86	90.43
MobileFaceNets [21]	0.99	439.8	99.55	-	-	-	96.07	-	-
PocketNetS-256 [14]	0.99	587.24	99.66	95.50	88.93	93.34	96.35	89.31	91.33
PocketNetS-128 [14]	0.92	587.11	99.58	95.48	89.63	94.21	96.10	89.44	91.62
ShuffleFaceNet 0.5x [22]	0.5	66.9	99.23	-	-	92.59	93.22	-	-
EdgeFace - S ($\gamma = 0.5$) **(ours)**	3.65	306.11	99.78	**95.71**	**92.56**	95.81	96.93	**93.58**	**95.63**
EdgeFace - XS ($\gamma = 0.6$) **(ours)**	1.77	154	**99.73**	95.28	**91.82**	94.37	96.00	**92.67**	**94.85**

Các Backbone của MobileFaceNet

Model	Kiến trúc lõi	Params	Ưu điểm	Nhược điểm
Sphere20/36	Standard Conv + Residual	24-34M	Accuracy cao nhất (95%+)	Quá nặng cho camera
MobileNetV1	Depthwise Separable Conv	0.22-0.36M	Siêu nhẹ, inference nhanh	Không có skip connection
MobileNetV2	Inverted Residual Block	0.46-2.29M	Cân bằng tốt, có residual	Hơi phức tạp hơn V1
MobileNetV3	Inverted Residual + SE + H-Swish	1.25-3.52M	Kiến trúc NAS tối ưu	Phức tạp nhất

```
Standard Conv (ResNet/VGG/Sphere):
- 1 bộ lọc 3×3 chà lên TẤT CẢ kênh cùng lúc
- Params = in_ch × out_ch × 3 × 3
- VD: Conv(64→128) = 64 × 128 × 9 = 73,728 params  ← QUÁ NẶNG

Depthwise Separable Conv (MobileNet):
- Tách 2 bước: (1) lọc spatial riêng, (2) trộn channel sau
- Params = in_ch × 9 + in_ch × out_ch
- VD: DWSep(64→128) = 576 + 8,192 = 8,768 params   ← GIẢM 8.4 LẦN!
```



MobileFaceNet dựa trên backbone MobileNetV1

- Kiến trúc gồm 3 stage nối tiếp cộng 1 lớp output GDC
- Có một kỹ thuật đặc trưng của MobileNet là Width multiplier
- Width multipiler giúp giảm thiểu khối lượng tính toán

Width Multiplier (α) hoạt động ra sao?

1. Depthwise Separable Convolutions:

- Tách convolution thành 2 bước riêng biệt
- Pointwise convolution (1x1) chịu ảnh hưởng trực tiếp từ số channels
- Giảm channels → giảm parameters rất hiệu quả

2. Design mục tiêu efficiency:

- MobileNet được thiết kế để mỏng nhẹ
- Width multiplier là tool để điều chỉnh độ mỏng/light này

3. Empirical validation:

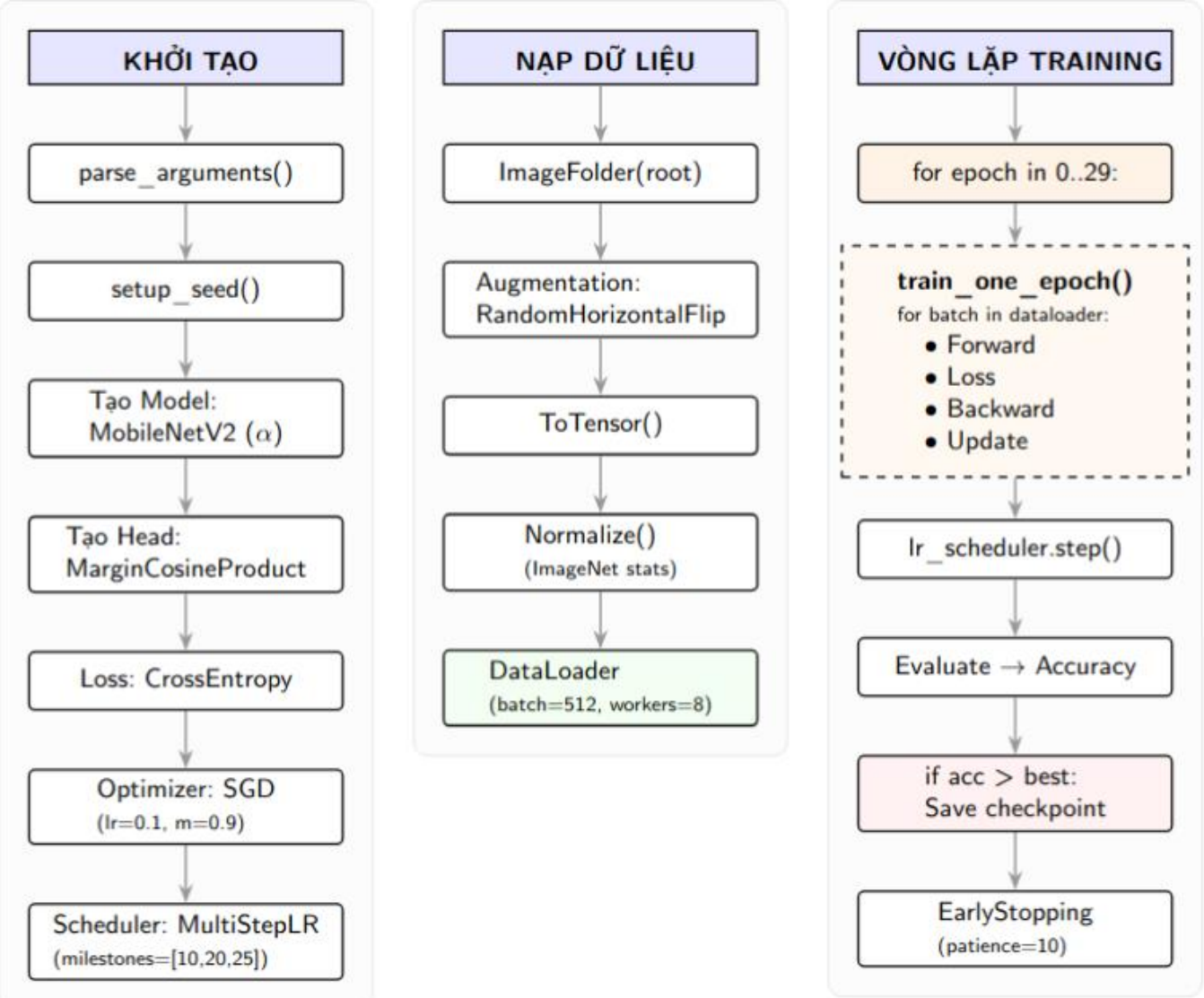
- Paper đã test với nhiều width_mult values
- 0.25, 0.5, 0.75, 1.0 đều hoạt động tốt với trade-off khác nhau

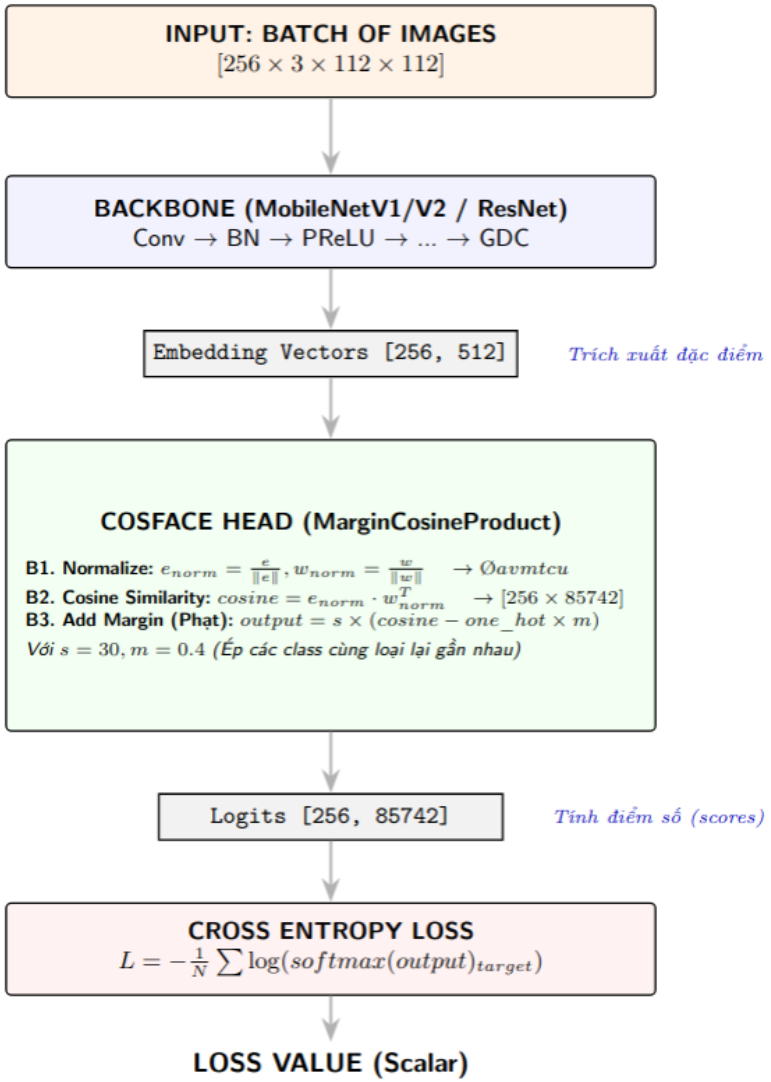
```
filters = [32, 64, 128, 256, 512, 1024]                    # Gốc
filters = [_make_divisible(f * width_mult) for f in filters] # Sau khi ép

# width_mult = 0.25 → filters = [8, 16, 32, 56, 112, 224]
# width_mult = 0.18 → filters = [8, 16, 24, 48, 96, 184]
```

width_mult	filters thực tế	Params	ONNX Size
1.0 (Full)	[32, 64, 128, 256, 512, 1024]	~3.2M	~12 MB
0.25	[8, 16, 32, 56, 112, 224]	~0.36M	1.39 MB
0.18	[8, 16, 24, 48, 96, 184]	~0.22M	0.87 MB

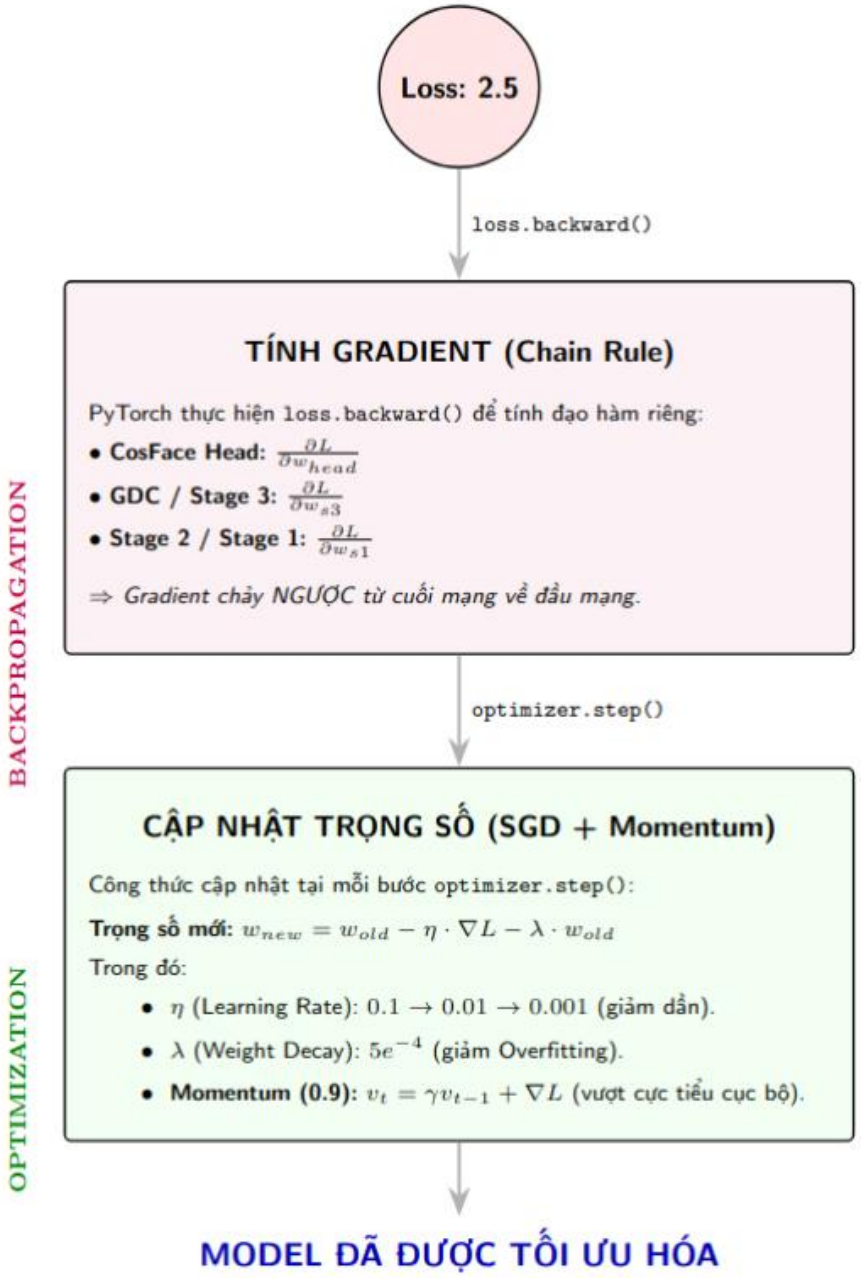
PIPELINE TRAINING





FORWARD PASS

- 1 BATCH ẢNH ĐI QUA HỆ THỐNG



Backward Pass — Model "sửa sai"

- Sơ đồ mô tả việc đạo hàm được tính ngược từ lớp cuối cùng (Head) về lớp đầu tiên (Conv1), từ đó weight được cập nhật

Khía cạnh	TRAINING	INFERENCE (Triển khai)
Mục đích	Dạy model phân biệt 85,742 người	Nhận diện 1 người cụ thể từ DB
Input	5.8M ảnh đã align sẵn 112×112	Video stream từ camera real-time
Face Detection	KHÔNG CẦN (ảnh đã crop sẵn)	CẦN (RetinaFace/InsightFace)
Face Alignment	KHÔNG CẦN (đã align sẵn)	CẦN (5-point landmark → transform)
Augmentation	CÓ (Flip, Normalize)	KHÔNG CÓ
CosFace Head	CÓ (85,742-way so sánh)	**KHÔNG CÓ** (Bỏ hoàn toàn!)
Loss + Backward	CÓ (tính loss, cập nhật weight)	**KHÔNG CÓ** (chỉ Forward)
Output	Loss value + accuracy	Embedding 512-D → so với DB
BatchNorm	mode train (tính running stats)	mode eval (dùng running stats)

Learning Similarity

HUST

hust.edu.vn fb.com/dhbkhn

HUST

THANK YOU !

hust.edu.vn fb.com/dhbkhn